

Notes on Geometric Deep Learning

Yashar Honarmandi

March 24, 2025

Abstract

This is a summary of the topic of Geometric Deep Learning. I was introduced to the topic when attending MLPrague in 2024 and found the concept highly appealing. Unfortunately the talk introducing the topic was utterly horrendous - the speaker's understanding was only displayed at a superficial level and nothing tangible came out of it. That is why I am doing this revisit.

Contents

1	Fundamental Concepts	1
---	----------------------	---

1 Fundamental Concepts

Supervised Learning Supervised learning deals with sets of iid observations (x_i, y_i) . The first element is called the data and the second is called the label, and these live in data space $\mathcal{X}$ and label space $\mathcal{Y}$ respectively. The data and labels are sampled from a probability distribution P defined on $\mathcal{X} \times \mathcal{Y}$.

Furthermore, the labels are generated from the data by some unknown function f , the estimation of which is the goal of supervised learning. The estimate is chosen from some parametrized class $\mathcal{F}$, which consists of members f_θ . The performance of each choice is evaluated based on expected performance on samples from P as measured by a loss function L . That is, the performance $\mathcal{R}(\tilde{f})$ is given by

$$\mathcal{R}(\tilde{f}) = \left\langle L(\tilde{f}(X), f(X)) \right\rangle_P. \quad (1)$$

The Curse of Dimensionality The curse of dimensionality refers to the set of phenomena where obtaining an adequate and regular fit to a dataset generally requires a number of samples that increases exponentially with the data dimensionality. Given that many machine learning applications concern high-dimensional data, this is a serious obstacle to doing machine learning.

Breaking the Curse Neural networks can break the curse of dimensionality by imposing sparsity on the weights, thus providing regularity. This comes, however, at the cost of making assumptions about the target function, which is undesirable. Geometric deep learning can instead provide a different source of regularity by analyzing structure and geometry in the data.

The Geometry of Data To start analysing data in terms of geometry, we will decouple the data domain as follows: We take it that $\mathcal{X}$ is composed of some domain Ω and some signal which maps points in Ω to some other domain $\mathcal{C}$. That is,

$$\mathcal{X} = \mathcal{X}(\Omega, \mathcal{C}) = \{x : \Omega \rightarrow \mathcal{C}\}. \quad (2)$$

The canon example for this is an $m \times n$ -size RGB image. In this case we would have $\Omega = \mathbb{Z}_m \times \mathbb{Z}_n$ and $\mathcal{C} = \mathbb{R}^3$ (or some other three-dimensional domain). In that case, a particular piece of data (a specific image) is a particular signal which describes how you need to distribute your RGB values to form a particular image.

The purpose of doing this is to separate out the geometry of the data. When considering for instance mirror or translation symmetry in an image, that only applies to the spatial part of the image, not the color part. Another thing it achieves is to separate out a part with the structure of a vector space. For instance, we can define an inner product between the RGB values of two different pixels to determine whether they are similar. Adding a measure μ on Ω , we can extend that to define an inner product between signals according to

$$\langle x_1 | x_2 \rangle = \int_{\Omega} d\mu(\omega) \langle x_1(\omega) | x_2(\omega) \rangle. \quad (3)$$

Geometric Priors There are two principles which we will refer to as our geometric priors: symmetry and scale separation.

Symmetry will here refer to the physical notion of symmetry, and thus include both discrete and continuous symmetries. Separating out the geometric part Ω of the data domain allows us to work directly with its symmetries.

Scale separation is the notion that phenomena at different scales can be treated separately. For instance, returning to the image analogy, scale separation entails that instead of always treating the minutia of a sample image, we can apply coarse-graining operations such as pooling without losing essential information.

Symmetry A symmetry is a transformation that leaves some property of the object it acts on invariant. These generally constitute a group.

Symmetry and Data Relevant to us is symmetry as it pertains to data. Acting on the domain it is defined as a mapping $\Omega \rightarrow \Omega$ with a particular symmetry group element $\mathbf{g}$ as a parameter in a way that preserves group closure. The new element is denoted as $u \rightarrow \mathbf{g}.u$ and closure gives that subsequent action by another group element $\mathbf{h}$ maps to $(\mathbf{h}\mathbf{g}).u$. Examples here include translations or rotations of Ω .

A group acting on Ω has a natural action on signals on Ω . Denoting such a signal as f we define

$$(\mathbf{g}.f)(u) = f(\mathbf{g}^{-1}u). \quad (4)$$

This definition satisfies closure. An example would be rotating images on a pixel grid. Note that this definition is linear in f , and so any group has a representation on the signal output space. That is,

$$(\mathbf{g}.f)(u) = \rho(\mathbf{g})f(u). \quad (5)$$

What is the distinction between the two? It is quite subtle, and you need to keep the domain and the signal separate. One way to imagine it is as follows: Suppose that you want to rotate a square image 90° clockwise. How do you go about it? One way to do it is to fix the pixels and rotate the grid counter-clockwise. Another way to do it is to rotate the pixels while fixing the grid. These two possibilities represent the two different cases respectively.

Invariants and Equivariants A function $f : \mathcal{X}(\Omega) \rightarrow \mathcal{Y}$ is invariant under a group $\mathfrak{G}$ if $f(\mathbf{g}.x) = f(x) \forall \mathbf{g} \in \mathfrak{G}$.

A function $f : \mathcal{X}(\Omega) \rightarrow \mathcal{X}(\Omega)$ may have another property, namely equivariance. This is defined as f satisfying $f(\mathbf{g}.x) = \mathbf{g}.f(x) \forall \mathbf{g} \in \mathfrak{G}$.

What are some examples of these two properties? Translation invariance is a typical example of the former. Given an image classification problem, the true label function is invariant under translations of the image, and so are convolutional neural networks that try to approximate it. Shift equivariance is found in the layers of a convolutional neural network, which typically have linear activation functions and thereby change in the same way as the input on a group action. This shift equivariance is broken by pooling layers in between.